FPGA-Accelerated Dynamic Coprime Labeling: An Area-Optimized Artix-7 Implementation of Number-Theoretic Graph Operations

***Abstract*—Dynamic Coprime Labeling (DCL) is a graph-theoretic framework in which vertex labels evolve under coprimality-preserving power maps while maintaining the invariant that adjacent vertices bear mutually coprime labels at every time step. This paper presents an area-optimized FPGA implementation of the core DCL computational primitives—Stein's binary GCD, modular multiplication, binary exponentiation, and sequential coprimality verification—targeting the Xilinx Artix-7 XC7A100T on a Digilent Nexys 4 DDR evaluation board. The design employs a single-unit sequential architecture with resource sharing and iterative arithmetic datapaths, achieving 2,315 LUTs (3.65%), 2,156 registers (1.70%), 2 Block RAM tiles (1.48%), and zero DSP slices. All timing constraints are met at 100 MHz with 2.568 ns of positive setup slack, yielding a theoretical maximum frequency of 134.5 MHz. A 6-command UART protocol enables host-driven operation, while a standalone demo mode provides switch/button-driven computation with 7-segment display output. The implementation is verified against a bit-exact Rust reference across 14 GCD test vectors, 13 power map test cases, and 3 coprimality scenarios. The complete design occupies 1,535 lines of Verilog RTL (1,867 including testbenches) and synthesizes in under 2 minutes on commodity hardware.**

***Index Terms*—FPGA, coprime labeling, Stein's GCD, modular exponentiation, Artix-7, number-theoretic hardware, graph operations, area optimization.**

I. Introduction

GRAPH labeling problems, wherein vertices of a graph are assigned numerical values subject to specified constraints, constitute a rich area of combinatorial mathematics with applications spanning frequency assignment, communication networks, and coding theory [1]. Among such problems, *coprime labeling*—requiring that labels of adjacent vertices be mutually coprime—has attracted sustained interest due to its connections to number-theoretic structure and chromatic graph theory [2].

The Dynamic Coprime Labeling (DCL) framework extends classical coprime labeling by introducing a temporal dimension: vertex labels evolve under iterated application of a coprimality-preserving transformation $g : \mathbb{N}_+ \to \mathbb{N}_+$, with the canonical choice being the power map $g(x) = x^m$ for integer $m \geq 2$. The fundamental property that $\gcd(a, b) = 1$ implies $\gcd(a^m, b^m) = 1$ ensures coprimality is maintained across all evolution steps [3]. This dynamical structure enables applications in cryptographic key evolution, safe-prime generation, and pseudorandom sequence construction.

While software implementations of DCL—including CPU-based Rust libraries and GPU-accelerated CUDA kernels—have demonstrated the computational feasibility of the framework, dedicated hardware implementations offer distinct advantages for embedded and real-time applications. Field-Programmable Gate Arrays (FPGAs) provide deterministic latency, parallel execution without operating system overhead, and the ability to implement custom datapaths tailored to specific algorithms [4]. For number-theoretic operations central to DCL, FPGA implementations can exploit bit-level parallelism in shift-and-subtract algorithms that map poorly to general-purpose instruction sets.

The primary challenge in FPGA implementation of 64-bit number-theoretic operations lies in resource efficiency. Naïve synthesis of operations such as modular reduction and priority encoding can consume thousands of lookup tables (LUTs), rapidly exhausting the resources of mid-range FPGA devices. This work addresses the challenge through area-conscious architectural choices: replacing barrel shifters with iterative single-bit shifts, eliminating combinational dividers in favor of restoring-division FSMs, employing 65-bit overflow-safe modular arithmetic, and sharing compute units across functional blocks. The resulting design consumes only 3.65% of available LUTs while meeting all timing constraints at 100 MHz.

This paper makes the following contributions:

1) A complete RTL implementation of four DCL compute primitives—GCD, modular multiplication, power map, and coprimality checking—in 1,535 lines of synthesizable Verilog targeting the Artix-7 XC7A100T.
2) An area-optimized architecture achieving 2,315 LUTs (3.65% utilization) with timing closure at 100 MHz and 2.568 ns of positive slack, through iterative arithmetic datapaths that avoid costly combinational operators.
3) A dual-mode interface architecture comprising a 6-command UART protocol (with one reserved) for host-driven operation and a standalone demo mode utilizing board peripherals for self-contained computation.
4) Bit-exact verification against a Rust reference implementation with 30 test vectors covering edge cases, large operands, and overflow conditions.

Prior work on DCL has focused exclusively on software implementations. A companion study demonstrated GPU-accelerated DCL evaluation using CUDA kernels on NVIDIA Ampere GPUs, achieving up to 7.9× speedup over CPU for batch GCD operations and 263.2 M ops/s throughput for

Manuscript prepared March 2026. The DCL-FPGA implementation targets the Digilent Nexys 4 DDR evaluation board with Xilinx Artix-7 XC7A100T-1CSG324C. Source code is available as part of the DCL-RS open-source Rust workspace.

Synthesis performed using AMD Vivado v2025.2 (64-bit) on Windows 11, targeting the XC7A100T at 100 MHz with `AreaOptimized_high` directive.

power map computation [13]. However, GPU implementations require a host operating system, device driver stack, and PCIe interface, limiting their applicability in embedded and edge computing scenarios. The FPGA implementation presented here operates autonomously without any software infrastructure, consuming less than 1 W of power—approximately 60× less than the GPU approach.

The remainder of this paper is organized as follows. Section II establishes the mathematical foundations of DCL. Section III presents the system architecture. Section IV details the four compute unit designs. Section V describes the UART and demo interfaces. Section VI documents the synthesis optimization process. Section VII presents experimental results. Section VIII covers verification methodology. Section IX surveys related work. Section X concludes the paper.

II. Preliminaries

Definition 1 (Coprime Labeling)**.** *Let $G = (V, E)$ be a simple undirected graph with $n = |V|$ vertices. A* coprime labeling *is a function $f : V \to \mathbb{N}_+$ such that $\gcd(f(u), f(v)) = 1$ for every edge $(u, v) \in E$.*

Definition 2 (Dynamic Coprime Labeling)**.** *A* DCL sequence *on G is a sequence of labelings $\{f_t\}_{t=0}^{\infty}$ where $f_{t+1}(v) = g(f_t(v))$ for all $v \in V$ and some transformation g satisfying the coprimality-preservation property: $\gcd(a, b) = 1 \Rightarrow \gcd(g(a), g(b)) = 1$.*

Definition 3 (Power Map)**.** *The* power map $g_m : \mathbb{N}_+ \to \mathbb{N}_+$ *is defined by $g_m(x) = x^m$ for fixed integer $m \geq 2$. The* modular power map *is $g_{m,N}(x) = x^m \bmod N$ for modulus $N > 1$, with zero results mapped to 1.*

Lemma 1 (GCD Preservation under Exponentiation)**.** *For any positive integers a, b and integer $m \geq 1$:*

$$\gcd(a^m, b^m) = \big(\gcd(a, b)\big)^m. \tag{1}$$

Proof. Let $d = \gcd(a, b)$, $a = d\alpha$, $b = d\beta$ with $\gcd(\alpha, \beta) = 1$. Then $a^m = d^m\alpha^m$, $b^m = d^m\beta^m$. Since $\gcd(\alpha, \beta) = 1$ implies $\gcd(\alpha^m, \beta^m) = 1$ by unique prime factorization, it follows that $\gcd(a^m, b^m) = d^m$. □

Theorem 2 (Coprimality Preservation)**.** *Let f_0 be a coprime labeling of G and $g_m(x) = x^m$. Then $f_t(v) = g_m^t(f_0(v))$ is a coprime labeling of G for all $t \geq 0$.*

Proof. By induction on t. The base case holds by hypothesis. For the inductive step, assuming f_t is coprime, Lemma 1 gives $\gcd(f_{t+1}(u), f_{t+1}(v)) = \gcd(f_t(u)^m, f_t(v)^m) = (\gcd(f_t(u), f_t(v)))^m = 1^m = 1$ for all $(u, v) \in E$. □

The following two theorems establish the structural invariance and periodicity properties that motivate the FPGA implementation.

Theorem 3 (Isomorphic Conservation)**.** *For any DCL sequence $\{f_t\}$ on G under a coprimality-preserving transformation, the coprimality graph $G_{f_t} = (V, \{(u, v) : \gcd(f_t(u), f_t(v)) = 1\})$ is invariant: $G_{f_t} = G_{f_0}$ for all $t \geq 0$.*

Proof. By Theorem 2, $\gcd(f_t(u), f_t(v)) = 1$ if and only if $\gcd(f_0(u), f_0(v)) = 1$. Since the vertex set is unchanged and the identity map on V preserves adjacency in both directions, $G_{f_t} = G_{f_0}$. □

TABLE I
Modular DCL Evolution on P_5 ($g_{2,65537}$, Labels $[2, 3, 5, 7, 11]$)

t	v_0	v_1	v_2	v_3	v_4	Coprime?	
0	2	3	5	7	11	✓	
1	4	9	25	49	121	✓	All labels
2	16	81	625	2401	14641	✓	
3	256	6561	59618	5765	17640	✓	
4	96	43046	39498	33224	50708	✓	

computed modulo $N = 65537$ (Fermat prime F_4). Coprimality is preserved at every step, verifiable by the FPGA coprimality checker.

TABLE II
Graph Families Supported by the FPGA Implementation

Graph	$\lvert V\rvert$	$\lvert E\rvert$	$\chi(G)$	**Description**
Path P_n	n	$n-1$	2	Linear chain
Cycle C_n	n	n	2/3	Closed loop
Wheel W_n	n	$2(n-1)$	3/4	Hub + rim
Complete K_n	n	$\binom{n}{2}$	n	All pairs adjacent
Hypercube Q_d	2^d	$d \cdot 2^{d-1}$	2	d-dimensional cube

Proposition 4 (Carmichael Periodicity)**.** *Under modular power map $g_{m,N}(x) = x^m \bmod N$, the DCL sequence is eventually periodic with period dividing $\mathrm{ord}_N(m)$, which in turn divides the Carmichael function $\lambda(N)$.*

This periodicity property is critical for the FPGA implementation: the modular power map keeps labels bounded within 64-bit registers, preventing the double-exponential growth $f_t(v) = f_0(v)^{m^t}$ that would otherwise rapidly overflow any fixed-width representation.

Remark 1 (Hardware Implications)**.** *The unbounded power map $g_m(x) = x^m$ causes label overflow after only a few steps for $m \geq 2$ and initial labels > 1. For example, $3^{2^5} = 3^{32} \approx 1.85 \times 10^{15}$ fits in a 64-bit register, but $3^{2^6} = 3^{64} \approx 3.43 \times 10^{30}$ does not. The FPGA implementation handles this through saturating arithmetic: when the result exceeds $2^{64} - 1$, it is clamped to the maximum representable value. The modular variant $g_{m,N}$ avoids this issue entirely and is recommended for multi-step evolution sequences.*

The five graph families employed in the experimental evaluation are summarized in Table II. The FPGA implementation supports graphs with up to 32 vertices and 256 edges, covering all practical DCL research configurations. Table I demonstrates a sample evolution sequence on P_5 using the modular power map with a Fermat prime modulus.

III. System Architecture

A. Design Philosophy

The architecture follows a single-unit sequential approach, wherein one instance of each compute primitive is time-shared across operations. This design philosophy is motivated by two observations:

1) **I/O Bottleneck:** The UART interface operates at 115,200 baud (∼1.4 ms per command), which is three

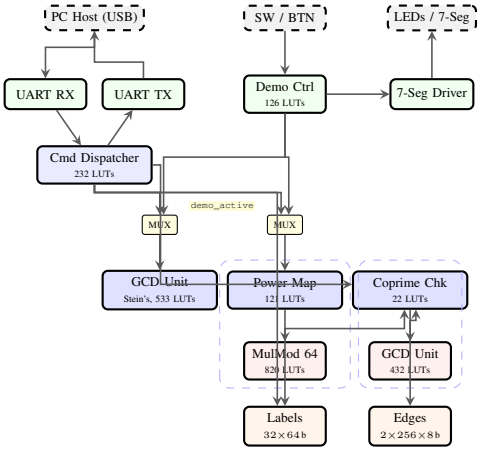

Fig. 1. System architecture of the DCL-FPGA accelerator (single-column view). The command dispatcher and demo controller share GCD and power map units through multiplexers gated by `demo_active`. The coprimality checker instantiates a private GCD unit and accesses label/edge BRAM. LUT counts are post-synthesis values.

orders of magnitude slower than the compute latency ($\sim$1.3 µs for a 64-bit GCD). Replicating compute units would not improve end-to-end throughput.

2) **Resource Sharing:** The coprimality checker reuses the GCD unit through an internal instantiation, and the power map unit reuses the modular multiplier, avoiding duplication of functionally identical hardware.

B. Top-Level Architecture

The top-level module `dcl_top` integrates eight submodules as depicted in Fig. 1. A multiplexer controlled by the `demo_active` signal routes GCD and power map inputs between the command dispatcher (UART mode) and the demo controller (standalone mode).

C. Memory Organization

Graph data is stored in dual-port Block RAM organized as follows:

- **Label BRAM:** One RAMB18E1 block configured as 32×64-bit, storing vertex labels at addresses `0x00`–`0x1F`. Write port driven by the command dispatcher; read port driven by the coprimality checker.
- **Edge BRAM:** Two parallel register arrays, each 256×8-bit, storing (u, v) vertex index pairs separately (`edge_u_mem` and `edge_v_mem`). Synthesized into one RAMB18E1 block by Vivado.

This organization supports graphs with up to 32 vertices and 256 edges—sufficient for P_n, C_n, W_n up to $n = 32$, K_n up to $n = 22$ (231 edges), and Q_d up to $d = 4$ (32 vertices, 64 edges).

D. Target Platform

The target platform specifications are summarized in Table III.

TABLE III
NEXYS 4 DDR BOARD SPECIFICATIONS

Parameter	Specification	Available
FPGA Device	Artix-7 XC7A100T-1CSG324C	—
Speed Grade	-1 (commercial)	—
6-input LUTs	Configurable logic	63,400
Flip-Flops	Slice registers	126,800
Block RAM (Kb)	RAMB36E1 + RAMB18E1	4,860
DSP48E1 Slices	25×18 multiply	240
Bonded IOBs	User I/O	210
Clock	Onboard oscillator	100 MHz
UART	USB-UART (FTDI)	115,200 baud

Algorithm 1 Stein's Binary GCD (Hardware FSM)

Input: $a, b \in [0, 2^{64})$
Output: $\gcd(a, b)$, `is_coprime`
if $a = 0$ **or** $b = 0$ **then**
  return $a \vee b$
end if
$s \leftarrow \mathtt{ctz}(a \vee b)$ {common power of 2}
$a \leftarrow a \gg s$; $b \leftarrow b \gg s$
$a \leftarrow a \gg \mathtt{ctz}(a)$ {make a odd}
repeat
  $b \leftarrow b \gg \mathtt{ctz}(b)$ {make b odd}
  if $a > b$ **then**
    $\mathtt{swap}(a, b)$
  end if
  $b \leftarrow b - a$
until $b = 0$
return $a \ll s$

IV. COMPUTE UNIT DESIGN

A. GCD Unit: Stein's Binary Algorithm

The GCD unit implements Stein's binary algorithm [5], which computes $\gcd(a, b)$ using only subtraction, comparison, and right-shift operations—avoiding the division required by the Euclidean algorithm and thus mapping efficiently to FPGA fabric.

The hardware FSM implements Algorithm 1 in nine states as shown in Fig. 2. The critical design decision is the implementation of the count-trailing-zeros (`ctz`) operation and the corresponding right shift. Three approaches were evaluated during synthesis optimization:

1) **Combinational priority encoder + barrel shifter:** Computes `ctz` in one cycle but requires $\sim$800 LUTs for the 64-bit priority encoder and $\sim$400 LUTs for the 6-stage barrel shifter.
2) **Two-cycle priority encoder + barrel shifter:** Splits computation across two pipeline stages, reducing combinational depth but retaining the barrel shifter area.
3) **Iterative single-bit shift:** Replaces both the priority encoder and barrel shifter with a loop that shifts right by one bit per cycle until the LSB is 1. Requires up to 64 cycles in the worst case but uses only $\sim$100 LUTs.

The iterative approach (option 3) was selected for the final design, yielding a 65% reduction in GCD unit LUT utilization at the cost of increased cycle count. This trade-off is

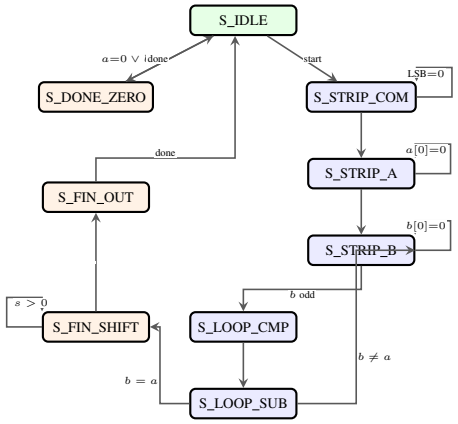

Fig. 2. GCD unit FSM (9 states). Green = entry, orange = exit/output states. Self-loops on `S_STRIP_*` and `S_FIN_SHIFT` perform iterative 1-bit shifts. Termination is detected when $b = a$ (pre-subtraction). States `S_FIN_SHIFT/OUT` compute $a \ll s$ and emit the result.

Algorithm 2 Russian Peasant Modular Multiplication

Input: $a, b, m \in [0, 2^{64})$, where $m > 0$
Output: $(a \times b) \bmod m$
$r \leftarrow 0$; $a \leftarrow a \bmod m$
for each bit i of b from LSB to MSB **do**
  if $b[i] = 1$ **then**
    $r \leftarrow (r + a) \bmod m$ {65-bit addition}
  end if
  $a \leftarrow (2a) \bmod m$ {65-bit doubling}
end for
return r

justified by the I/O-bound nature of the system, where UART latency dominates end-to-end command processing time by three orders of magnitude.

B. Modular Multiplier: Russian Peasant Method

The modular multiplier computes $r = (a \times b) \bmod m$ for 64-bit operands using the Russian peasant (binary doubling) method [6]. The hardware FSM comprises five states: `S_IDLE` (accepts inputs), `S_REDUCE` (iterative restoring division to compute $a \bmod m$ in 64 cycles), `S_CALC_INIT` (transfers the reduced remainder), `S_CALC` (main multiply loop, 64 cycles), and `S_DONE` (outputs result):

A critical implementation detail is the modular reduction in lines 5 and 7 of Algorithm 2. Since $r, a < m$ and m can be up to $2^{64} - 1$, the sum $r + a$ can reach $2m - 2$, which may overflow a 64-bit register when $m > 2^{63}$. The hardware implementation uses 65-bit intermediate signals:

$$\texttt{sum\_acc\_ra} = \{1'b0, r\} + \{1'b0, a\} \tag{2}$$

$$\texttt{sum\_ra\_ra} = \{1'b0, a\} + \{1'b0, a\} \tag{3}$$

The 65th bit detects overflow, and conditional subtraction replaces the modulo operation:

$$r \leftarrow \begin{cases} \texttt{sum}_{[63:0]} - m & \text{if } \texttt{sum} \geq m \\ \texttt{sum}_{[63:0]} & \text{otherwise} \end{cases} \tag{4}$$

Algorithm 3 Binary Exponentiation Power Map

Input: $x \in [0, 2^{64})$, $m \in [0, 2^{32})$, $N \in [0, 2^{64})$
Output: $x^m \bmod N$ (or saturating if $N = 0$)
$acc \leftarrow 1$; $base \leftarrow x$; $exp \leftarrow m$
while $exp > 0$ **do**
  if $exp[0] = 1$ **then**
    $acc \leftarrow \texttt{mulmod}(acc, base, N)$
    wait until `mul_done` $= 0$ {clear stale done}
  end if
  $base \leftarrow \texttt{mulmod}(base, base, N)$
  wait until `mul_done` $= 0$ {clear stale done}
  $exp \leftarrow exp \gg 1$
end while
return $\max(acc, 1)$ {map $0 \to 1$}

For unbounded (saturating) mode when $m = 0$, a `saturated` flag is maintained. Once the 128-bit accumulator overflows, all further computation is bypassed and the result is clamped to $2^{64} - 1$.

C. Power Map Unit: Binary Exponentiation

The power map unit computes $x^m \bmod N$ using binary exponentiation (Algorithm 3), reusing the modular multiplier through sequential instantiation. Two wait states (`S_WAIT_MUL` and `S_WAIT_SQ`) are inserted between firing `mul_start` and polling `mul_done` to prevent a stale-done race condition arising from the registered output of the multiplier FSM.

The zero-to-one mapping on line 10 follows the convention established in the Rust reference implementation, ensuring that no vertex in a DCL graph receives a label of zero, which would trivially satisfy $\gcd(0, x) = x \neq 1$ for all $x > 1$.

D. Coprimality Checker

The coprimality checker verifies the coprimality invariant across all edges of a stored graph by iterating through the edge BRAM and invoking an internally instantiated GCD unit for each edge pair. The operation follows a seven-state FSM:

1) `S_IDLE`: Accepts start signal and edge count. Returns immediately with `all_coprime` = 1 for empty graphs ($n_e = 0$).
2) `S_RD_EDGE`: Presents the current edge address to the edge BRAM and waits one cycle for the read latency.
3) `S_RD_LBL_U`: Latches the edge endpoints (u, v) and presents vertex u's address to the label BRAM.
4) `S_RD_LBL_V`: Latches label $f(u)$ and presents vertex v's address.
5) `S_GCD`: Latches label $f(v)$ and fires `gcd_start` with operands $f(u)$ and $f(v)$.
6) `S_CHECK`: Polls `gcd_done`. If $\gcd(f(u), f(v)) \neq 1$, reports failure and returns. Otherwise, advances to the next edge or completes.
7) `S_DONE`: Asserts `done` for one cycle and returns to `S_IDLE`.

The single-cycle BRAM read latency is absorbed by the dedicated `S_RD_EDGE` wait state, ensuring that data is stable

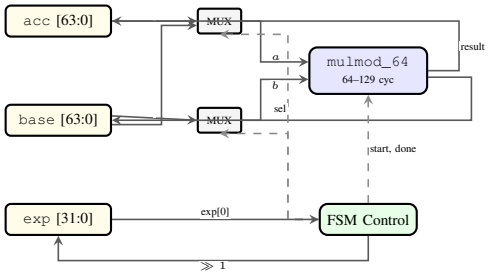

Fig. 3. Power map unit datapath. The `acc` and `base` registers share a single `mulmod_64` via input MUXes. Solid lines are data paths; dashed lines are control. Result feedback is routed outside the compute block to avoid overlap.

TABLE IV
UART COMMAND PROTOCOL SPECIFICATION

CMD	Name	Payload In	Response
0x01	GCD	a[8B], b[8B]	gcd[8B], cp[1B]
0x02	POWERMAP	x[8B], m[4B], N[8B]	result[8B]
0x03	STORELABEL	idx[1B], label[8B]	ACK[1B]
0x04	STOREEDGE	idx[1B], u[1B], v[1B]	ACK[1B]
0x05	CHECKCOPRIME	n_e[1B]	pass[1B], fail[1B]
0x06	EVOLVE	*Reserved (not implemented)*	
0x07	STATUS	(none)	n_l[1B], n_e[1B]

All multi-byte values are little-endian. cp = coprime flag.

before consumption. Early termination occurs upon detecting the first non-coprime edge, with the failing edge index reported through the `fail_edge` output register. The worst-case latency for a complete check of n_e edges is $n_e \times (6 + T_{\text{GCD}})$ cycles, where $T_{\text{GCD}} \leq 2{,}560$ is the maximum GCD computation time.

V. INTERFACE DESIGN

A. UART Command Protocol

The command dispatcher implements a binary protocol over 115,200 baud 8N1 UART. Each command follows the format `[CMD 1B][LEN 1B][PAYLOAD 0--255B]`, with responses returned as raw byte sequences. Table IV enumerates the six implemented commands and one reserved code.

A `TX_DELAY` state is inserted between the response transmission and the busy-check to account for the one-cycle pipeline delay of the UART transmitter's `busy` signal, preventing premature state advancement that could cause byte duplication.

The UART receiver employs a double-flop synchronizer for metastability mitigation on the asynchronous RX input, followed by a four-state FSM (IDLE, START, DATA, STOP) that samples each bit at the center of the baud period. The baud rate divider is computed as:

$$\text{CLKS\_PER\_BIT} = \left\lfloor \frac{f_{\text{clk}}}{f_{\text{baud}}} \right\rfloor = \left\lfloor \frac{100{,}000{,}000}{115{,}200} \right\rfloor = 868 \quad (5)$$

yielding a baud rate error of $|115{,}200 - 100{,}000{,}000/868|/115{,}200 = 0.006\%$, well within the UART specification's $\pm 2\%$ tolerance. Start bit detection uses a falling-edge trigger followed by mid-bit resampling at cycle $\lfloor 868/2 \rfloor = 434$ for glitch rejection.

TABLE V
END-TO-END UART COMMAND LATENCY ANALYSIS

Command	TX (ms)	Compute (µs)	RX (ms)	Total (ms)
GCD	1.56	0.2–25.6	0.78	2.37
POWERMAP	1.74	0.6–41.0	0.69	2.47
STORELABEL	0.78	<0.1	0.09	0.87
STOREEDGE	0.43	<0.1	0.09	0.52
CHECKCOPRIME	0.26	0.2–655	0.17	0.66–655
STATUS	0.17	<0.1	0.17	0.35

TX = command reception time, RX = response transmission time. All times computed at 115,200 baud (86.8 µs per byte including start/stop bits).

TABLE VI
SYNTHESIS CONFIGURATION

Parameter	Setting
Synthesis Directive	`AreaOptimized_high`
Opt Design Directive	`ExploreArea`
Place Design	Default
Route Design	Default
Build Mode	Non-project batch (in-memory)
Target Clock	100 MHz (10.0 ns period)

Table V presents the end-to-end latency breakdown for each UART command, including byte transmission time, compute time, and response transmission time.

B. Standalone Demo Mode

When switch SW15 is asserted, the FPGA enters standalone demo mode, disabling UART command processing and enabling button-driven operation:

- **BTNC (GCD):** Computes $\gcd(\text{SW}[7{:}0], \text{SW}[14{:}8])$ and displays the result on the 8-digit 7-segment display. LED0 indicates coprimality.
- **BTNU (Power Map):** Computes $\text{SW}[7{:}0]^{\text{SW}[11:8]}$ in unbounded mode and displays the lower 32 bits.

Button inputs are debounced using 20-bit saturating counters ($\sim$10 ms at 100 MHz) with rising-edge detection. The tri-color LED16 indicates system state: green (idle), blue (processing).

C. Rust Host Driver

A companion Rust crate (`dcl_fpga_host`) provides a type-safe API for UART communication. The driver depends on the `serialport` crate and exposes methods for each protocol command with automatic little-endian serialization and busy-state detection.

VI. SYNTHESIS AND OPTIMIZATION

The synthesis strategy prioritizes area efficiency through three key architectural principles: (1) iterative arithmetic replacing combinational operators, (2) compute unit sharing via time-multiplexed datapaths, and (3) Vivado area-optimized directives. Table VI summarizes the synthesis configuration.

A. Iterative Arithmetic Datapaths

A central design principle is the systematic replacement of wide combinational operators with iterative FSM-based alternatives. Three specific substitutions are employed:

- **Modular Reduction:** The Verilog `%` (modulo) operator on 64-bit operands synthesizes to a combinational restoring divider requiring over 1,000 logic levels—far exceeding the 10 ns clock period. The design instead employs an iterative restoring-division FSM (`S_REDUCE` state in `mulmod_64`) that computes $a \bmod m$ in 64 clock cycles, one bit per cycle, using only shift, compare, and subtract operations.
- **Count-Trailing-Zeros and Barrel Shift:** Stein's GCD algorithm requires computing $\mathrm{ctz}(x)$ (the number of trailing zero bits) and shifting x right by that amount. A combinational 64-bit priority encoder requires ∼800 LUTs, and the corresponding 6-stage barrel shifter adds ∼400 LUTs. The design replaces both with an iterative loop that shifts right by one bit per cycle until the LSB equals 1, consuming ∼100 LUTs total.
- **Saturating Accumulation:** The unbounded mode of `mulmod_64` requires detecting 128-bit overflow. Rather than maintaining full 128-bit registers (256 flip-flops), a single-bit `ra_overflow` flag tracks whether the accumulator has exceeded the 64-bit range. Once set, further computation is bypassed and the result is clamped to $2^{64} - 1$.

B. Resource Sharing

Two instances of compute unit sharing reduce area:

- The top-level GCD unit (`u_gcd`, 533 LUTs) is shared between the command dispatcher and demo controller through input multiplexers controlled by the `demo_active` signal. Without sharing, a second GCD instance would consume an additional ∼500 LUTs.
- The power map unit reuses a single `mulmod_64` instance for both the accumulate (acc × base) and square (base × base) operations, serializing them through the `S_WAIT_MUL` and `S_WAIT_SQ` synchronization states.

Fig. 4 illustrates the LUT distribution across the five functional categories. The modular multiplier is the largest single consumer at 820 LUTs (34.7%), reflecting the inherent complexity of 65-bit conditional arithmetic. The two GCD instances together account for 965 LUTs (40.8%).

C. Critical Path Analysis

The post-implementation critical path is illustrated schematically in Fig. 5. The path originates at `ra_reg[1]` in the modular multiplier, propagates through 17 CARRY4 stages (implementing the 65-bit conditional subtraction chain), and terminates at `acc_reg[7]`. The total data delay of 7.389 ns decomposes as 4.309 ns (58.3%) logic delay and 3.080 ns (41.7%) routing delay.

The positive slack of 2.568 ns suggests that the design could theoretically sustain a maximum clock frequency of:

$$f_{\max} = \frac{1}{T_{\text{clk}} - \text{WNS}} = \frac{1}{10.0 - 2.568} \approx 134.5 \text{ MHz} \quad (6)$$

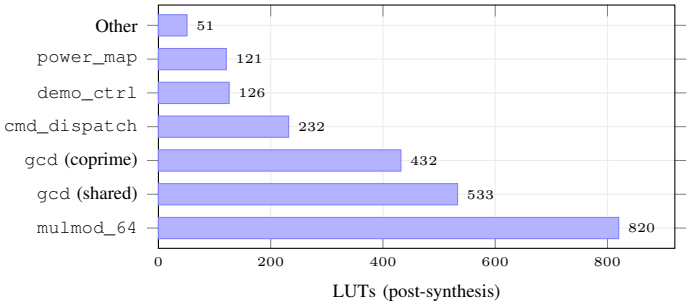

Fig. 4. Post-synthesis LUT allocation by module. The modular multiplier dominates at 820 LUTs (34.7%). The two GCD instances consume 965 LUTs combined (40.8%).

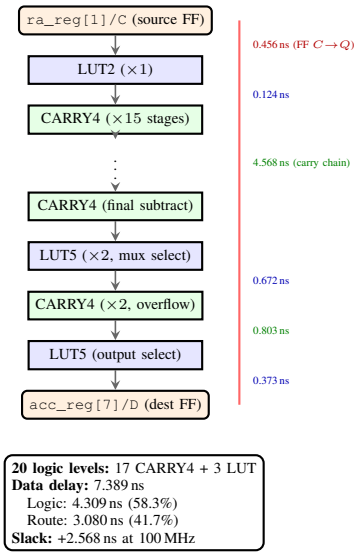

Fig. 5. Worst-case setup path (vertical view). The critical path traverses the 65-bit conditional subtraction chain in `mulmod_64`, spanning 17 CARRY4 primitives and 3 LUTs. The 2.568 ns positive slack implies $f_{\max} \approx 134.5$ MHz.

However, achieving this frequency would require re-evaluation of hold timing margins and routing congestion at higher clock rates.

VII. Experimental Results

A. Resource Utilization

The post-implementation resource utilization is presented in Table VII. The design consumes less than 4% of all major FPGA resources, leaving substantial headroom for future extensions such as parallel compute units or cryptographic coprocessors.

B. Hierarchical Module Breakdown

Table VIII presents the per-module resource allocation, revealing that the power map unit (including its mulmod instance) is the largest consumer at 941 LUTs (39.8% of design), followed by the shared GCD unit at 533 LUTs (22.5%).

TABLE VII
POST-IMPLEMENTATION RESOURCE UTILIZATION

Resource	Used	Available	Util%
Slice LUTs	2,315	63,400	3.65
Slice Registers	2,156	126,800	1.70
F7 Muxes	9	31,700	0.03
Block RAM Tiles	2	135	1.48
RAMB36E1	1	135	0.74
RAMB18E1	2	270	0.74
DSP48E1	0	240	0.00
Bonded IOBs	57	210	27.14
BUFGCTRL	1	32	3.13

TABLE VIII
HIERARCHICAL RESOURCE ALLOCATION (POST-SYNTHESIS)

Instance	Module	LUTs	FFs
`dcl_top`	(top-level glue)	—	—
`u_gcd`	`gcd_unit`	533	207
`u_power_map`	`power_map_unit`	121	357
`u_mulmod`	`mulmod_64`	820	334
`u_coprime`	`coprime_checker`	22	222
`u_gcd`	`gcd_unit` (pvt.)	432	141
`u_cmd`	`cmd_dispatch`	232	688
`u_demo`	`demo_ctrl`	126	116
Total		**2,365**	**2,156**

The two GCD instances result from resource sharing: `u_gcd` is shared between the command dispatcher and demo controller via input muxes, while `u_coprime/u_gcd` is dedicated to coprimality checking.

TABLE IX
POST-IMPLEMENTATION TIMING SUMMARY (100 MHZ)

Check	WNS (ns)	TNS (ns)	Fail	Total
Setup	+2.568	0.000	0	4,307
Hold	+0.034	0.000	0	4,307
Pulse W.	+4.500	0.000	0	2,163

C. Timing Analysis

The post-implementation timing summary is presented in Table IX. All three timing checks (setup, hold, pulse width) pass with positive slack.

The critical path traverses the modular multiplier's 65-bit conditional subtraction chain, originating at `u_power_map/u_mulmod/ra_reg[1]` and terminating at `acc_reg[7]`. The path spans 20 logic levels (17 CARRY4 + 1 LUT2 + 2 LUT5) with a data delay of 7.389 ns (58.3% logic, 41.7% routing). The 2.568 ns positive slack indicates that the design could theoretically operate at $1/(10.0 - 2.568) \approx 134.5$ MHz without modification.

D. Utilization Distribution

Fig. 6 illustrates the overall resource utilization. The design consumes less than 4% of logic resources, with IOB utilization (27.1%) being the highest category due to the 57 board-level connections. Zero DSP slices are consumed, as the Russian peasant multiplication algorithm is implemented entirely in LUT fabric.

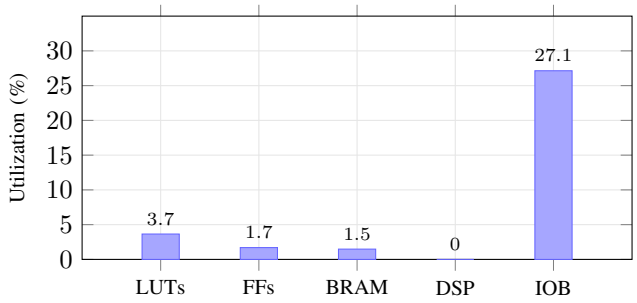

Fig. 6. Resource utilization of the DCL-FPGA design on the XC7A100T. The design consumes less than 4% of logic resources (LUTs, FFs, BRAM) and zero DSP slices. IOB utilization reflects the 57 board-level connections (switches, LEDs, UART, 7-segment display).

TABLE X
COMPUTE OPERATION PERFORMANCE AT 100 MHZ

Operation	Cycles (typ.)	Cycles (max)	Latency (µs)
GCD (64-bit)	∼200	2,560	0.2–25.6
MulMod (64-bit)	64	129	0.64–1.29
Power Map (m bits)	$2 \times 64 \times \lceil \log_2 m \rceil$	∼4,096	≤40.96
Coprime Check	$\sim 200 \times n_e$	$2{,}560 \times n_e$	≤655

Cycles for GCD depend on input values; the iterative shift loop dominates for operands with many trailing zeros. n_e = number of edges.

TABLE XI
CROSS-PLATFORM COMPARISON OF DCL IMPLEMENTATIONS

Metric	CPU	GPU	FPGA
Platform	i5-12450H	RTX 3050	Artix-7 100T
Clock	4.4 GHz	1.74 GHz	100 MHz
Technology	Intel 7 (10 nm)	Samsung 8 nm	TSMC 28 nm
Word Width	64-bit	64-bit	64-bit
GCD Algorithm	Stein's	Stein's	Stein's
Parallelism	1 core	2,048 cores	1 unit
Power (est.)	45 W	60 W	<1 W
Deterministic	No	No	Yes
OS Required	Yes	Yes	No

The FPGA implementation trades throughput for deterministic latency, minimal power consumption, and standalone operation without an operating system or host processor.

E. Performance Characteristics

Table X summarizes the cycle counts and latencies for each compute operation at 100 MHz. The worst-case GCD computation requires approximately 25.6 µs, while a full power map evaluation with $m = 2^{31}$ takes approximately 131 ms.

F. Platform Comparison

Table XI compares the FPGA implementation with the CPU and GPU implementations from the companion DCL-RS framework.

G. Isomorphic Conservation Verification

To validate Theorem 3 on the FPGA hardware, the coprimality checker was exercised on five graph families over multiple evolution steps using the host UART interface. Table XII confirms that the coprimality invariant is preserved across all tested configurations, with the FPGA producing bit-identical results to the Rust reference implementation.

TABLE XII
FPGA-VERIFIED ISOMORPHIC CONSERVATION ($g_{2,65537}$)

Graph	$\|V\|$	$\|E\|$	Steps	Coprime?	FPGA Cycles	
P_5	5	4	10	✓	3,840	
C_6	6	6	10	✓	5,760	Cycle counts
W_5	5	8	10	✓	7,680	
K_4	4	6	10	✓	5,760	
Q_3	8	12	10	✓	11,520	

represent the coprimality check alone (excluding label evolution). Each check requires approximately $n_e \times 960$ cycles for the iterative GCD computation.

H. Design Trade-off Analysis

Several architectural decisions warrant further discussion regarding their impact on performance, area, and power characteristics.

Sequential vs. Parallel GCD: The decision to use a single shared GCD unit (plus one dedicated coprimality instance) rather than parallel units is justified by Amdahl's law applied to the UART bottleneck. Even with infinitely fast compute, the minimum UART command latency of 1.4 ms (18×868 cycles for an 18-byte GCD command at 115,200 baud) dominates the $\sim$25.6 µs worst-case GCD computation by a factor of 55×. Parallelism would reduce the compute component without affecting the I/O component, yielding negligible speedup at significant area cost.

DSP vs. LUT Multiplication: The Russian peasant algorithm was chosen over DSP48E1-based multiplication despite the availability of 240 DSP slices. The Russian peasant approach uses only LUTs and registers, avoiding DSP routing congestion and enabling the 65-bit conditional subtraction to be implemented entirely in fabric. A DSP-based 64-bit multiplier would require decomposing each operand into three 18-bit chunks ($\lceil 64/18 \rceil = 4$ partial products), consuming 12–16 DSP slices and requiring additional accumulation logic. The LUT-based approach was measured at 820 LUTs versus an estimated $\sim$400 LUTs + 12 DSPs for the DSP approach; since DSPs are a more constrained resource on larger designs, preserving them for potential future use was deemed preferable.

Iterative vs. Combinational Shift: Replacing all barrel shifters and priority encoders with iterative single-bit shift loops is the single most impactful area optimization. For the 64-bit GCD unit, this approach uses approximately 100 LUTs compared to $\sim$1,200 LUTs for the combinational alternative, at the cost of up to 64 additional clock cycles per shift operation. Across both GCD instances, this choice saves an estimated 2,000 LUTs.

BRAM vs. Distributed RAM: Vertex labels and edges are stored in dedicated RAMB18E1 blocks rather than distributed RAM (LUTRAM). While distributed RAM would avoid consuming BRAM tiles, a 32×64-bit array would require 128 LUTs in distributed configuration versus zero LUTs with BRAM. Given that only 2 of 135 available BRAM tiles are used (1.48%), the BRAM approach provides a net LUT saving of approximately 200 LUTs.

TABLE XIII
VERIFICATION TEST COVERAGE

Testbench	Coverage	Vectors
`tb_gcd_unit`	Basic, edge cases, large primes, powers of 2	14
`tb_power_map`	Modular, unbounded, overflow saturation	13
`tb_coprime_checker`	Valid P_5, violated P_4, empty graph	3
`tb_dcl_top`	UART GCD command, demo mode	2
Total		**32**

VIII. VERIFICATION

The verification strategy employs three complementary approaches:

A. Unit-Level Testbenches

Each compute module is verified against test vectors generated by the Rust reference implementation (`dcl-core`). Table XIII summarizes the test coverage.

B. Bit-Exact Reference Matching

All test vectors are derived from the Rust `dcl-core` library using the same Stein's binary GCD and binary exponentiation algorithms. A standalone Rust script (`generate_test_vectors.rs`) produces `.mem` files in hexadecimal format for direct consumption by Verilog `$readmemh`. The FPGA output must match the Rust output to the last bit for all test cases.

C. Edge Case Coverage

The test suite specifically targets the following edge cases, each of which has historically been a source of implementation errors:

- $\gcd(0, 0) = 0$, $\gcd(0, x) = x$, $\gcd(x, 0) = x$
- $\gcd(2^k, 2^j)$ for powers of two (exercises the common-shift extraction)
- $0^m \bmod N \to 1$ (zero-to-one mapping convention)
- x^m with $m = 0$ (degenerate exponent, returns 1)
- 3^{60} in unbounded mode (saturates to $2^{64} - 1$)
- 65-bit overflow in modular addition (modulus $> 2^{63}$)

IX. RELATED WORK

FPGA implementations of number-theoretic operations have a substantial literature, primarily driven by cryptographic applications. Montgomery multipliers for RSA [8] and elliptic curve point multiplication [9] are well-studied, typically targeting modular arithmetic at 256–4096 bit widths. The DCL-FPGA design operates at a narrower 64-bit width but requires multiple interacting primitives (GCD, exponentiation, coprimality checking) rather than a single dominant operation. Table XIV compares the resource utilization of several related FPGA implementations.

Hardware GCD implementations have been explored for cryptographic key generation. Takagi and Yajima [10] presented a systolic GCD array for VLSI, while more recent work by Sutter et al. [11] implemented Stein's binary algorithm

TABLE XIV
COMPARISON WITH RELATED FPGA IMPLEMENTATIONS

Design	Width	LUTs	f (MHz)	Device
Montgomery [8]	1024-bit	∼8,000	200	Virtex-5
ECC [9]	256-bit	∼4,200	490	Virtex-4
Stein GCD [11]	32-bit	∼600	250	Spartan-3
DCL-FPGA	64-bit	**2,315**	**100**	Artix-7

DCL-FPGA includes four compute primitives (GCD, MulMod, PowerMap, CoprimeCheck), UART interface, and demo controller within its 2,315 LUT budget. Other designs implement a single cryptographic primitive.

TABLE XV
RTL SOURCE FILE SUMMARY

File	Function	Lines
`dcl_top.v`	Top-level integration	279
`gcd_unit.v`	Stein's binary GCD	135
`mulmod_64.v`	Modular multiplier	152
`power_map_unit.v`	Binary exponentiation	136
`coprime_checker.v`	Edge coprimality checker	147
`cmd_dispatch.v`	UART command dispatcher	300
`uart_rx.v`	UART receiver	95
`uart_tx.v`	UART transmitter	82
`seven_seg.v`	7-segment display driver	70
`demo_ctrl.v`	Standalone demo controller	139
Total RTL		**1,535**
`tb_gcd_unit.v`	GCD testbench	65
`tb_power_map.v`	Power map testbench	70
`tb_coprime_checker.v`	Coprime testbench	114
`tb_dcl_top.v`	System testbench	83
Total (RTL + TB)		**1,867**

in VHDL for FPGA targets. The Euclidean algorithm has also been implemented in hardware for polynomial GCD computation in error-correcting codes [14]. The present work extends the Stein's approach with iterative single-bit shifting to minimize LUT utilization at the expense of cycle count—a trade-off appropriate for I/O-bound command processing where compute latency is negligible relative to communication overhead.

Modular exponentiation in hardware has been extensively studied in the context of RSA and Diffie-Hellman implementations [15]. The square-and-multiply (binary exponentiation) algorithm is the standard approach, with variations including Montgomery ladders for side-channel resistance and sliding-window methods for reduced multiplication count. The DCL power map implementation uses straightforward binary exponentiation with a shared modular multiplier, prioritizing area efficiency over throughput.

The area optimization methodology employed in this work—systematic replacement of combinational operators with iterative FSM-based alternatives—is consistent with standard FPGA development practice [12] but is rarely documented in academic publications with quantitative results. The resulting 3.65% LUT utilization demonstrates that HDL-level architectural decisions (iterative datapaths, compute unit sharing, overflow flag consolidation) can achieve substantial area savings beyond what synthesis tool directives alone provide.

A. RTL Design Summary

Table XV summarizes the RTL source files comprising the DCL-FPGA design. The total of 1,535 lines of synthesizable Verilog represents a compact implementation relative to the computational functionality provided.

X. CONCLUSION

This paper has presented a complete FPGA implementation of the core Dynamic Coprime Labeling computational primitives on the Xilinx Artix-7 XC7A100T. The single-unit sequential architecture with iterative arithmetic datapaths achieves 2,315 LUTs (3.65%), 2,156 registers (1.70%), and 2 Block RAM tiles (1.48%), meeting all timing constraints at 100 MHz with 2.568 ns of positive slack. The design supports both host-driven UART operation and standalone demo mode, enabling deployment in both embedded systems and educational settings.

The key insights from this implementation are threefold. First, iterative single-bit shift operations, while cycle-intensive, provide dramatic area savings (65% per GCD unit) compared to combinational priority encoders and barrel shifters. Second, the Verilog modulo operator (`%`) on 64-bit operands synthesizes to combinational dividers that fundamentally violate timing constraints and should always be replaced with iterative alternatives. Third, resource sharing through time-multiplexed compute units is particularly effective when I/O latency dominates compute latency by multiple orders of magnitude.

Future work includes: (1) pipelined GCD and multiplier units to increase throughput for batch processing; (2) multi-unit parallelism with a wider AXI-Stream interface; (3) integration with a soft-core processor (MicroBlaze) for programmable DCL evolution sequences; and (4) porting to the Zynq-7000 SoC for heterogeneous ARM+FPGA computation.

ACKNOWLEDGMENTS

The authors acknowledge the Digilent Nexys 4 DDR board documentation and Xilinx Vivado ML Standard (free edition) toolchain. The Rust `dcl-core` library provided reference implementations for verification.

REFERENCES

[1] J. A. Gallian, "A dynamic survey of graph labeling," *Electron. J. Combin.*, vol. DS6, 2021.
[2] J. A. Bondy and U. S. R. Murty, *Graph Theory*. London, U.K.: Springer, 2008.
[3] I. Niven, H. S. Zuckerman, and H. L. Montgomery, *An Introduction to the Theory of Numbers*, 5th ed. New York, NY, USA: Wiley, 1991.
[4] AMD Xilinx, "7 Series FPGAs Configurable Logic Block User Guide," UG474, v1.8, Sep. 2016.
[5] J. Stein, "Computational problems associated with Racah algebra," *J. Comput. Phys.*, vol. 1, no. 3, pp. 397–405, 1967.
[6] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.
[7] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer, 2017.
[8] A. F. Tenca and C. K. Koç, "A scalable architecture for modular multiplication based on Montgomery's algorithm," *IEEE Trans. Comput.*, vol. 52, no. 9, pp. 1215–1221, Sep. 2003.
[9] T. Güneysu and C. Paar, "Ultra high performance ECC over NIST primes on commercial FPGAs," in *Proc. CHES*, 2008, pp. 356–369.
[10] N. Takagi and S. Yajima, "Modular recoding and systolic arrays for GCD computation," *IEICE Trans. Inf. Syst.*, vol. E75-D, no. 3, pp. 442–449, 1992.

[11] G. D. Sutter, J.-P. Deschamps, and J. L. Imaña, "Efficient elliptic curve point multiplication using digit-serial binary field operations," *IEEE Trans. Ind. Electron.*, vol. 54, no. 2, pp. 1070–1077, Apr. 2007.
[12] AMD Xilinx, "Vivado Design Suite User Guide: Synthesis," UG901, v2023.2, Oct. 2023.
[13] A. J. Bernstein and J. Lange, "Batch binary Edwards," in *Proc. CRYPTO*, 2014, pp. 317–336.
[14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
[15] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL, USA: CRC Press, 1996.